

partie 4 : Les structures

MOOC Langage C INSA

Restriction : Ce document ne peut être utilisé que dans le cadre des cours de l'INSA de Toulouse

Source : Ce document est en partie extrait du livre : du langage C au C++ par Thierry Monteil, Vincent Nicomette, François Pompignac, Saturnino Hernando, Presses Universitaires du Midi ISBN 978-2-8107-0054-7

Comme pour un record, en Pascal ou Ada, il est possible de regrouper sous un seul nom des structures de données comprenant, en général, plusieurs variables de types différents, afin d'en faciliter le traitement.

1 Principes fondamentaux

1.1 Déclarations, définitions

La déclaration d'une structure de données utilise le mot réservé **struct**, suivi d'une liste entre accolades des champs - typés - de la structure. On a donc la forme générale :

```
struct
{
    /* corps */
};
```

On peut faire suivre le mot **struct** d'un nom, appelé étiquette. Par exemple :

```
struct Modele
{
    /* corps */
};
```

Ces déclarations ne réservent pas de place en mémoire (ce ne sont donc pas des définitions) : elles décrivent tout simplement un modèle de structure de données. Si la déclaration est étiquetée, on peut l'utiliser par la suite pour déclarer des variables selon l'exemple suivant :

```
struct Modele a, b, c, *pStructure;
```

qui définit les structures **a**, **b** et **c** de type **struct Modele**, ainsi qu'un pointeur **pStructure** qui ne peut recevoir que l'adresse d'une structure de même type. Le compilateur réserve donc en mémoire la place nécessaire aux variables **a**, **b**, **c** et **pStructure**. Si la structure n'est pas étiquetée, il faut faire simultanément la déclaration et la définition :

```
struct
{
    /* corps */
}
a, b, c, *pStructure;
```

Enfin, on peut aussi utiliser une étiquette dans la définition précédente ce qui permet de réutiliser le type `struct Modele` par la suite :

```
struct Modele
{
    /* corps */
} a, b, c, *pStructure;
```

Toutes ces définitions réservent de la place en mémoire.

1.2 Les champs de la structure

Les éléments du corps d'une structure sont appelés champs. Ils sont typés. Ainsi, la déclaration :

```
struct Entreprise
{
    char Sigle[10];
    char Adresse [40];
    int Telephone;
};
```

possède trois champs. Pour accéder à l'un des champs d'une structure, on peut utiliser l'opérateur `.` (point) de sélection de champs. Les instructions suivantes :

```
struct Entreprise Ingenieurs;

strcpy(Ingenieurs.Sigle,"INSA");
strcpy(Ingenieurs.Adresse,"Complexe Scientifique de Rangueil");
Ingenieurs.Telephone = 61559513;
```

affectent respectivement l'adresse de deux chaînes et la valeur d'un entier aux champs `Sigle`, `Adresse` et `Telephone` de la variable `Ingenieurs`.

1.3 Une structure n'est pas un tableau !

On peut initialiser une structure au moment de sa définition selon l'exemple suivant :

```
struct Entreprise Ingenieurs =
{
    "INSA"
    "Complexe Scientifique de Rangueil",
    61559513;
};
```

Cependant, une structure n'est pas assimilable à un tableau. Nous avons déjà dit que le type tableau n'existe pas ; ce n'est pas le cas du type structure. Ainsi le nom d'un tableau représente son adresse d'implantation, alors que le nom d'une structure telle que `Ingenieurs` est une variable. On ne peut pas directement affecter un tableau à un autre tableau, par contre on peut le faire entre structures de même type.

Ainsi :

```
char Tableau [40], Phrase [40];
struct Entreprise Ingenieurs, Facultes;

Tableau = Phrase;          /* INTERDIT, Tableau n'est pas une l-value ! */
Ingenieurs = Facultes;     /* Correct */
```

Dans cet exemple, la valeur de la variable `Facultes` est recopiée dans la variable `Ingenieurs`. Cette copie s'effectue automatiquement champ par champ. Nous reviendrons un peu plus loin sur ce type d'affectation.

1.4 Structures imbriquées

Les structures peuvent être emboîtées les unes dans les autres. Continuons en développant le même exemple :

```
struct Personnel
{
    unsigned int Age;
    struct Entreprise Ecole;
};
struct Personnel Directeur, Comptable;
```

Remarquez bien que `Ecole` n'est pas une variable structurée, mais un champ structuré, vide à priori. Même si vous l'aviez appelé `Ingenieurs` (défini et initialisé auparavant), ce champ reste local aux variables de type `struct Personnel`, et ne peut être confondu (du moins par le compilateur...) avec la variable `Ingenieurs`. Aussi ne vous recommandons-nous pas ce genre d'acrobatie. Autrement dit, évitez les homonymes ! Comment affecter le champ `Ecole` ? Vous pouvez le faire globalement en lui affectant la valeur d'une variable de même type structuré, par exemple :

```
Directeur.age= 55;
Directeur.Ecole = Ingenieurs;
```

Vous pouvez aussi utiliser l'opérateur point, qui associe les opérandes de gauche à droite :

```
Comptable.age = 28;
strcpy(Comptable.Ecole.Sigle,"INSA");
```

Cette façon d'emboîter les structures n'est pas souple. Si l'on a à manipuler des champs structurés volumineux, leur affecter une valeur champ par champ est très lourd à écrire. Si on fait une affectation globale de tels champs, ceci peut ralentir le programme. Aussi préfère-t-on, souvent, employer des pointeurs de structure :

```
struct Personnel
{
    ...
    ...
    ...
    struct Entreprise *pLocal;
};
```

Dans cet exemple, en modifiant simplement le contenu du champ `pLocal`, on peut accéder à n'importe quelle structure de modèle `Entreprise`. On n'a plus besoin que d'un seul exemplaire des structures de ce modèle, puisque toutes les structures de modèle `Personnel` n'ont besoin que de leur référence.

2 Structures et pointeurs

Si nous écrivons :

```
struct Modele
{
    int a;
    char c;
    double f;
} Var_Struct, *pStructure;
```

`pStructure` est un pointeur qui peut pointer sur une structure du modèle indiqué, mais qui n'est pas initialisé. Comme nous l'avons déjà vu, l'initialisation se fait, par exemple, ainsi :

```
pStructure = &Var_Struct;
```

si bien que `pStructure` pointe sur la variable `Var_Struct`. Pour accéder à un champ de la structure à partir de ce pointeur, il faut utiliser l'opérateur `->` (moins - supérieur, qui symbolise une flèche) :

```
pStructure -> c
```

Cette notation représente le champ `c` de la structure pointée par `pStructure`. Cette expression est une l-value de même type que le champ correspondant, ici un `char`. La forme :

```
(*pStructure).c
```

est équivalente, quoique moins employée. Les parenthèses sont nécessaires car le point est plus prioritaire que l'indirection (`*`).

3 Les tableaux de structures

3.1 Déclaration, initialisation

La déclaration

```
struct Mois
{
    char Nom[20];
    int Jours;
} TabMois [12];
```

définit un tableau appelé `TabMois` dont les 12 éléments sont des structures selon le modèle `Mois`. On pouvait aussi déclarer le modèle `Mois`, puis définir le tableau :

```
struct Mois TabMois [12];
```

Un tel tableau peut être initialisé à la définition :

```
struct Mois TabMois [] =
{
    { "Janvier", 31 },
    { "Fevrier", 28 },
    /* ... */
    { "Decembre", 31 }
};
```

Attention ! L'initialisation des tableaux multidimensionnés de structures est encore plus délicate.

3.2 Pointeur sur un tableau

Si l'on utilise un pointeur de structure pour parcourir un tel tableau, la dimension de la structure est automatiquement prise en compte lorsque l'on additionne ou retranche des entiers à ce pointeur :

```
struct Mois * pMoisCourant = TabMois;
/* pMoisCourant pointe sur la structure de rang 0,
   c'est a dire { "Janvier", 31 } */

pMoisCourant++;      /* pointe sur la structure de rang 1,
                       soit { "Fevrier", 28 } */
pMoisCourant += 10; /* pointe alors sur la structure de
                    rang 11, soit { "Decembre", 31 } */
```

On peut connaître la taille de la mémoire occupée par une structure grâce à l'opérateur `sizeof` :

```
int Taille;

Taille = sizeof (struct Mois);
```

`sizeof` fournit le nombre d'octets occupés par une structure du modèle indiqué.

Remarque : dans l'exemple précédent, `Mois` n'est pas un type, c'est `struct Mois` qui l'est. Aussi préfère-t-on souvent attribuer un nom à ce type grâce à `typedef` :

```
typedef struct Mois NouveauType;
NouveauType Un, Deux, Trois, *pSoleil;
```

Ici, `Un`, `Deux` et `Trois` sont des variables structurées selon le modèle `Mois`. On pouvait aussi écrire :

```
typedef struct Mois          /* Etiquette facultative */
{
    char Nom[20];
    int Jours;
} NouveauType;
```

3.3 Opérateurs, pointeurs, et structures

Les opérateurs `->` et `.` (point) sont avec les crochets `[]` et les parenthèses `()` les opérateurs les plus prioritaires : les relations qu'ils établissent sont très étroites. Pour illustrer cela, prenons l'exemple du tableau de structures suivant :

```
struct Exemple
{
    int Val [4];
    int * ptrVal;
} Tableau [3], * pStruct;

pStruct = Tableau;
```

`pStruct` reçoit l'adresse du tableau, il pointe donc sur la première structure `Tableau[0]`. Si on fait :

```
pStruct->ptrVal = pStruct->Val;
```

le champ `ptrVal` de la première structure du tableau pointe sur son autre champ : un tableau de quatre entiers. Remarquez que `Tableau` (ou `pStruct`) et `Tableau[0].Val` (ou `pStruct->ptrVal`) ont la même valeur, l'adresse d'implantation en mémoire, mais le compilateur C les distingue car `Tableau` est l'adresse d'une structure, alors que `Tableau[0].Val` est l'adresse d'un entier. Comme l'opérateur `->` est de plus haute priorité, écrire :

```
++pStruct->ptrVal;
```

est équivalent à

```
++(pStruct->ptrVal);          /* notation a preferer */
```

qui montre que ce n'est pas `pStruct` qui est incrémenté mais le champ `ptrVal`. Ce dernier, qui pointait sur le champ `Val[0]`, pointe maintenant l'entier contenu dans le champ `Val[1]`. Si on veut explicitement incrémenter `pStruct` pour passer à la structure suivante dans `Tableau`, il faut utiliser des parenthèses dans une expression de ce type :

```
((++ pStruct)->ptrVal = pStruct->Val;
```

`pStruct` pointe alors sur la structure `Tableau[1]`, dont le champ `ptrVal` pointe sur le premier élément du tableau du champ `Val`.

```
(pStruct++)->ptrVal;
```

accède à ce qui est pointé, puis incrémente `pStruct`. Les parenthèses ne sont pas nécessaires, mais elles sont préférables, toujours dans un souci de lisibilité.

```
*pStruct->ptrVal
```

est identique à :

```
*(pStruct->ptrVal)
```

et représente l'élément pointé par le champ `ptrVal`, un entier donc.

```
*pStruct->ptrVal++
(*pStruct->ptrVal)++
```

incrémentent cet entier, alors que :

```
*pStruct++.ptrVal ou
(*pStruct++).ptrVal
```

retournent la valeur de cet entier puis incrémentent `pStruct`. Ceci nous permet donc de parcourir un tableau de structures en ne sélectionnant qu'un seul champ.

4 Structure se référant à elle-même

Il est interdit de déclarer un modèle de structure se contenant elle-même, mais un de ses champs peut être un pointeur sur une structure de même type :

```
struct Liste
{
    int Ordre;
    double Valeur;
    struct Liste * Suivant;
};
```

Ainsi, on peut organiser un chaînage de structures identiques afin de faire du "list processing", c'est à dire construire des listes chaînées, des arbres, des piles, etc... Si le nombre de structures est inconnu a priori, il faut faire des allocations dynamiques de la mémoire. Il existe pour cela des fonctions d'allocation (`malloc()`, `calloc()`...) qui permettent d'obtenir de la mémoire durant l'exécution.

5 Structures et fonctions

5.1 Opérations sur les structures

Les seules opérations possibles avec des structures sont :

- le calcul de son adresse, à l'aide de l'opérateur `&`,
- l'accès à l'un de ses champs en utilisant les opérateurs `->` ou `*`,
- la recopie du contenu complet d'une structure dans une autre structure de même modèle (nous l'avons déjà vu) à condition que ses champs soient des variables scalaires.

5.2 Structures passées en argument

Puisque les structures sont des variables, elles peuvent être passées par valeur ou par référence. Cependant, pour une structure volumineuse, on préfère le passage par référence afin de ne pas encombrer la pile associée au programme, ni ralentir exagérément le programme. Quand on a une fonction ayant de nombreux arguments, il est plus facile de les enfermer dans une structure. Cela allègera l'écriture d'appel de cette fonction, mais la déclaration de la structure doit être globale à la fonction appelante et à la fonction appelée, comme on va le voir. Passer en argument une structure depuis une fonction appelante vers une fonction appelée présente des restrictions que nous symbolisons ici par quelques schémas, où `Appelante()` et `Appelée()` sont des fonctions.

1. La déclaration de la structure doit être externe (figure 1) :

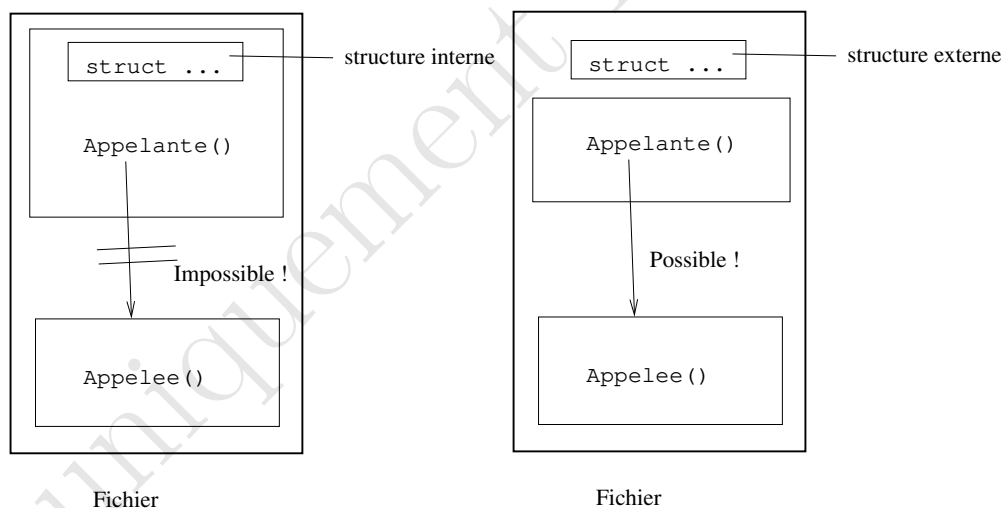


FIGURE 1 – Restriction d'appel

2. On ne peut pas importer un modèle de structure dans un fichier. Ceci est logique car une déclaration de ce type :

```
extern struct Modele MaStructure;
```

bien que tolérée par le compilateur, ne permet pas d'accéder aux divers champs, car ceux-ci ne sont pas explicitement déclarés (figure 2).

Si divers fichiers doivent employer des types structurés communs, la meilleure solution est celle de les déclarer dans un fichier d'en-tête (traditionnellement son nom possède le suffixe `.h`, de header), puis d'inclure ce fichier dans tous les fichiers où c'est utile. Une telle technique est schématisée dans la figure 3, où trois fichiers incluent le même fichier d'en-tête, `fic.h` :

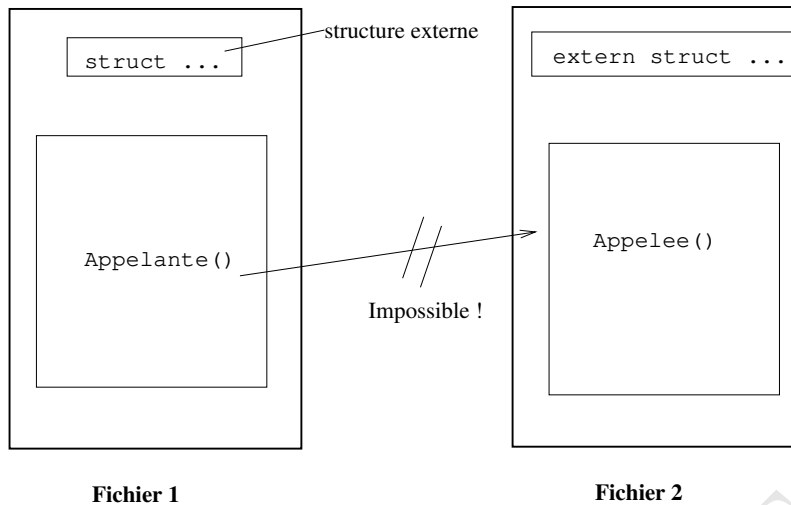


FIGURE 2 – Import d'un modèle de structure interdit

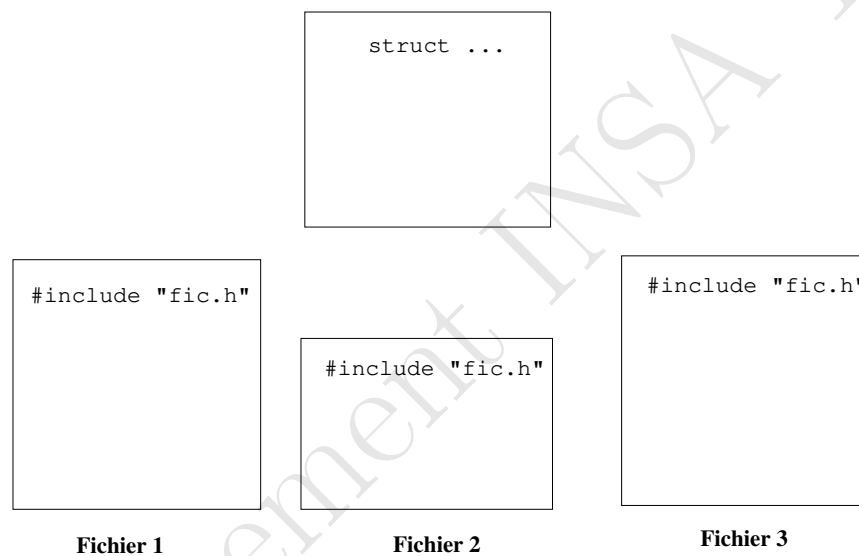


FIGURE 3 – Comment partager le même modèle de structure

5.3 Structure renvoyée par une fonction

Enfin, une fonction peut renvoyer la valeur d'une structure ou un pointeur sur une structure :

```
struct Modele Directe ();
struct Modele * Indirecte ();
```

6 Le constructeur typedef

6.1 Présentation de typedef

Nous avons déjà utilisé plusieurs fois `typedef`, et il est temps d'en expliquer le fonctionnement en détail, car ce mot réservé est particulièrement utilisé avec des structures. Nous savons que `typedef` permet d'attribuer un nom symbolique à un type simple ou composé. La syntaxe d'utilisation de `typedef` est facile, quoique quelquefois surprenante. Considérez une déclaration de variable (ou de tableau, de structure, de fonction, etc...). Si vous la faites précéder du mot réservé `typedef`, le nom de ce qui était une variable (ou tableau, fonction,...) devient le nom symbolique du type (simple ou composé) utilisé dans la déclaration, mais `typedef` ne réserve jamais de mémoire. Par exemple :


```
typedef unsigned char UCHAR;
typedef char PHRASE [80];
typedef int FONCTION (UCHAR, char *);
/* UCHAR, PHRASE, FONCTION sont de nouveaux types */

UCHAR LettreNSign = 0xff;
PHRASE Message = "Une chaine de moins de 80 caracteres...";
FONCTION Longueur;
int Retour;

Retour = Longueur (LettreNS, Message);
```

L'usage veut que le nouveau type soit défini en majuscules, mais ce n'est pas imposé. Ici, `LettreNSign` est une variable de type caractère non signé, `Message` est le nom d'un tableau de 80 caractères, et `Longueur` est le nom d'une fonction renvoyant un entier, et admettant deux paramètres. Dans cet exemple, `UCHAR` représente un type simple, mais `PHRASE` et `FONCTION` symbolisent des types composés. `typedef` est souvent utilisé pour alléger l'écriture de types compliqués :

```
typedef char * P_CHAR;
typedef P_CHAR TAB_DIX [10];
typedef TAB_DIX * PTR_TAB
typedef PTR_TAB (* FONCT) ();
FONCT Enfin;
```

On a donc :

- `P_CHAR` est le type pointeur de caractère.
- `TAB_DIX` est le type tableau de 10 éléments de type `P_CHAR`.
- `PTR_TAB` est le type pointeur sur un objet de type `TAB_DIX`.
- `FONCT` est le type pointeur sur une fonction renvoyant une valeur de type `PTR_TAB`.

Si bien que la variable `Enfin`, qui est du type `FONCT`, est en fait un pointeur sur une fonction renvoyant la valeur d'un pointeur sur un tableau de 10 pointeurs vers un caractère - ou une chaîne de caractères ! Avec un peu de courage, on pouvait aussi déclarer directement `Enfin` :

```
char * ( * ( * Enfin ) ( ) ) [10];
```

dont voici le type généralisé correspondant :

```
char * ( * ( * ) ( ) ) [ ];
  6  5  3  1  2  4
```

Pour lire un tel type composé, il faut se rappeler que parenthèses et crochets ont la même plus haute priorité et se lisent de gauche à droite. On part donc de la parenthèse de gauche. On y trouve emboîtée une seconde parenthèse, prioritaire par rapport à l'astérisque noté 3. On part donc de 1 : il s'agit d'un pointeur (1) sur une fonction (2) qui renvoie la valeur d'un pointeur (3) qui pointe sur un tableau (4) de pointeurs (5) de caractères (6) !

6.2 typedef et les structures

L'écriture de structures, tableaux de structures, structures emboîtées, etc... étant relativement lourde, l'utilisation de `typedef` peut rendre bien plus agréable l'écriture et la lecture d'un programme. Voici alors les diverses manières classiques d'utiliser `typedef` avec un type structuré, en créant un type appelé ici `NOUVEAU` :

1. Le nouveau type est défini grâce au modèle construit auparavant :

```
struct Modele
{
    /* corps */
};
typedef struct Modele NOUVEAU;
```

2. Le nouveau type est construit lors de la définition de la structure :

```
typedef struct {
    /* corps */
} NOUVEAU;
```

Ici, l'étiquette de la structure est inutile, mais l'usage veut qu'on l'insère malgré tout dans la construction, en utilisant le même nom que le nouveau type, précédé du caractère `_` :

```
typedef struct _NOUVEAU
{
    /* corps */
} NOUVEAU;
```

7 Autres organisations de données

Les organisations de données de type :

- champs de bits,
- unions,
- enum,

ont des configurations semblables à des structures. Voyons-les donc.

7.1 Champs de bits

Un champ de bits est une structure où chaque champ contient un nombre quelconque de bits. En guise d'exemple, la structure suivante représente l'état d'un registre associé à la gestion de la mémoire par pagination :

```
struct Simul_Registre
{
    unsigned char PG : 1;    /* 1 bit */
    unsigned char I  : 1;    /* 1 bit */
    unsigned char R  : 1;    /* 1 bit */
    unsigned char C  : 1;    /* 1 bit */
    unsigned char NB : 2;    /* 2 bits */
    unsigned char LRU: 2;    /* 2 bits */
} Reg;
```

On accède par exemple au bit du champ `R` de la variable `Reg` par la notation :

`Reg.R`

comme pour n'importe quelle autre structure.

On comprend que l'intérêt principal des champs de bits réside dans la compression des données en mémoire centrale ou de masse (fichiers, sauvegardes du contenu de registres, de mots d'état des drivers, de comptes-rendus de l'utilisation de périphériques, etc...). Voici à titre d'exemple comment est organisée la table de relocation d'un fichier objet pour station de travail SUN3 sous UNIX (Berkeley 4.2). Une telle table est un tableau de structures du modèle `reloc_info_68k` :

```

struct reloc_info_68k
{
    long r_address;                /* address which is relocated */
    unsigned int r_symbolnum :24, /* local symbol ordinal */
    r_pcrel :1,                    /* was relocated pc-relative already */
    r_length :2,                  /* 0= byte, 1=word, 2=long */
    r_extern :1,                  /* does not include value of sym referenced */
    r_basereel :1,                /* linkage table relative */
    r_jmptable :1,               /* pc-relative to jump table */
    r_relative :1,               /* relative relocation */
    :1;                          /* dummy */
};

```

Un bit, sans étiquette, a été rajouté afin que cette structure soit composée de 64 bits, et coïncide donc avec deux entiers. Si vous devez utiliser des champs de bits, consultez la documentation de votre compilateur pour savoir exactement comment il les implante en mémoire. Dans l'exemple ci-dessus, le champ `r_address` recouvre le premier entier (le type `long` a la même taille que le type `int`), le champ `r_symbolnum` correspond à trois octets en poids fort du deuxième entier de la structure, alors le bit de poids le plus faible du quatrième octet de cet entier est le bit d'alignement (ou bit de bourrage, `dummy`).

7.2 Unions

Comme une EQUIVALENCE en FORTRAN, une union est une variable qui peut contenir à différents moments des objets de types différents. La taille de ces objets peut être différente. Les unions fournissent donc un moyen de manipuler différents types de données dans un même emplacement de mémoire. La syntaxe de déclaration et d'appel est basée sur celle des structures :

```

union Nombre
{
    int Entier;
    double Reel;
} Ixe;

```

Dans cet exemple, `Ixe` est une variable dont la taille en mémoire (fixe) est à la fois capable d'accepter soit une valeur de type `int`, soit une de type `double`. Le compilateur a donc choisi la taille de la mémoire affectée à `Ixe` pour qu'elle puisse contenir le type le plus long, c'est-à-dire `double`.

Attention ! On ne peut ranger dans `Ixe` qu'une seule valeur à la fois. Ne confondez pas avec les structures : ces dernières ont en général plusieurs champs de types fixes, alors qu'une union n'a qu'un seul champ, mais le type de ce dernier est a priori variable. C'est au programmeur de savoir quel est le type de variable qui a été rangée dans l'union, puisque le langage C (comme on pouvait s'y attendre) ne fait aucune vérification de cohérence durant l'exécution.

Ainsi :

```
Ixe.Entier
```

est la variable entière, et

```
Ixe.Reel
```

est la variable double, placée au même endroit en mémoire..

Les unions peuvent aussi être employées à l'intérieur de tableaux ou de structures, et inversement. Une union dans une structure permet donc de faire un record possédant une partie variante comme en Pascal ou en Ada. Voici un exemple :

```

struct Modele                /* 4 champs */
{
    char * Nom;
    int Entier;
    int Type;
    union
    {
        int Val_Ent;
        double Val_Reel;
        char * Val_Ptr;
    } Champ_Union;
} Table [10];

```

Dans l'exemple ci-dessus :

`Table[7].Champ_Union.Val_Ent`

est, par exemple, une variable entière rangée dans `Champ_Union` de la 8ème structure de `Table`. De même :

`*Table[7].Champ_Union.Val_Ptr`

est le premier caractère de la chaîne pointée par le même `Champ_Union`. Une union peut donc servir à économiser de la mémoire en l'affectant à plusieurs variables. Elle permet aussi de reconsidérer la même valeur de diverses manières. Ainsi, voici l'exemple d'une simulation des huit registres de données du microprocesseur 68000. Ces registres peuvent, selon le cas - et en utilisant la terminologie de l'assembleur - être considérés comme des registres de mots longs (32 bits), des registres contenant deux mots (16 bits), ou des registres contenant quatre octets :

```

union Reg_Donnees
{
    char B [4];
    short W [2];
    long L ;
} D0, D1, D2, D3, D4, D5, D6, D7, D8;

D5.L = 0x1234ABCD;

```

Alors, `D5.B[0]` vaut `0x12`, `D5.B[2]` vaut `0xAB`, `D5.W[1]` vaut `0xABCD`.

Voici un (autre) exemple amusant :

```

/* Pour qu'un entier contienne sa propre adresse... */
union
{
    int Valeur;
    int * Reference;
} Objet;

int main ()
{
    Objet.Reference = &Objet.Valeur;
    printf ("\n valeur      : %10d", Objet.Valeur);
    printf ("\n reference : %10d\n", *Objet.Reference);
    return 0;
}

```

Exemple d'exécution :

```
valeur      :      131272
reference   :      131272
```

En fait, cet exemple revient à un forçage de type :

```
Objet.Valeur = (int) &Objet.Valeur;
```

7.3 Énumérations

En C, le mot réservé `enum` permet de construire un type énuméré, comme il en existe en Pascal ou en Ada. Par exemple :

```
enum Jours {lun, mar, mer, jeu, ven, sam, dim };
```

déclare par énumération le type `enum Jours`.

Selon exactement les mêmes règles de syntaxe que pour une structure, on peut définir des variables du type `enum Jours` (de manière analogue aux structures, `Jours` n'est ici aussi qu'une étiquette symbolisant un modèle) :

```
enum Jours Ouvrable, Ferie;
```

et les variables `Ouvrable` ou `Ferie`, peuvent prendre les valeurs de la définition :

```
Ouvrable = lun;
Ferie = dim;
```

`enum Jours` est en réalité un type entier, car le compilateur affecte la valeur 0 au premier membre de l'énumération, 1 au second membre, etc... On peut aussi leur affecter explicitement des valeurs entières particulières (à la définition seulement) :

```
enum
{
    plus = '+',
    moins = '-'
} Operateur;
```

Voici maintenant un exemple final, associé à `typedef` :

```
typedef enum { Vrai, Faux } BOOLEEN;

/* Fonction Egal() : renvoie Vrai ou Faux selon que les
valeurs passees en arguments sont egales ou differentes */

BOOLEEN Egal ( int X, int Y )
{
    return ( (BOOLEEN) abs(X - Y) );
}
```

Remarquez que dans cet exemple, on a pris la convention inverse à celle du C pour la représentation des booléens : ici 0 signifie Vrai. Notez aussi la coercition dans le `return`, nécessaire : en effet, en l'absence de ce forçage, `Egal()` renverrait bien 0 s'il y avait égalité, mais, dans le cas contraire, ne renverrait pas 1, mais la valeur absolue de la différence entre `X` et `Y`.